Sixteenth Implementation Iteration: Space Allocation

with External Request Loading and Data Controller

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this sixteenth implementation iteration is to improve the architecture of the prototype by moving request definitions to an external CSV source and centralizing all data access responsibilities in a dedicated controller class.

In previous iterations, users and spaces were already loaded from external files, but requests were still created manually inside the main program logic. In addition, the main program directly interacted with lower-level data loading and writing components.

In this iteration, requests are also loaded from an external CSV file, and a new `DataController` is introduced to serve as a single gateway for loading and exporting system data. This improves maintainability and helps isolate the internal business logic from storage-specific details.

2. Changes from Iteration 15

Compared with the fifteenth iteration, the following changes were introduced:

- Added a new external file: `requests.csv`
- Added support for loading one-time and recurring requests from CSV
- Introduced a new `DataController` class
- Refactored the main program so that it uses the data controller instead of directly calling data loading and writing utilities
- Added support for exporting request processing results to an external CSV file
- Expanded the number of sample users, spaces, and requests to create richer interaction scenarios

3. Scenario

The prototype starts by loading users, spaces, and requests from external CSV files. The loading process is handled through a centralized `DataController`. After loading is completed, the allocation service processes all requests and produces allocation results.

At the end of execution, the system exports:

- accepted allocations to `allocations.csv`
- full processed request outcomes to `request_results.csv`

This means the main program no longer depends directly on the details of how CSV files are parsed or written.

4. Modules and Their Tasks

A. Data Controller Module

- Serve as the single entry point for loading system data
- Load users, spaces, and requests through one controller interface
- Export generated allocations and processed request results
- Clean up dynamically allocated request and space objects after execution

B. Data Loading Module

- Load users from `users.csv`
- Load spaces from `spaces.csv`
- Load one-time and recurring requests from `requests.csv`
- Convert external CSV values into internal domain objects

C. Request Management Module

- Receive requests from external input instead of hardcoded main logic
- Represent both one-time and recurring requests using the existing request model
- Preserve existing request fields such as requester, participant count, feature, building, and priority

D. Rule Evaluation and Allocation Module

- Continue evaluating requests through the existing rule engine and facade
- Approve or reject requests based on implemented rules
- Create allocations when rule evaluation succeeds

E. Output Export Module

- Export accepted allocations to `allocations.csv`
- Export request outcomes, including rejection reasons, to `request_results.csv`

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
12	Abstract Request	Class + Lab + Meet	Request polymorphism
13	Same	Same	Rule Engine + Facade
14	Same	Same	Structured user roles + role rule
15	Same	Same	Request priority model
16	Externalized requests	Class + Lab + Meet	DataController + CSV request loading

6. Updated Class Diagram

Figure 1 shows the updated class diagram including the `DataController` and the request result export support.

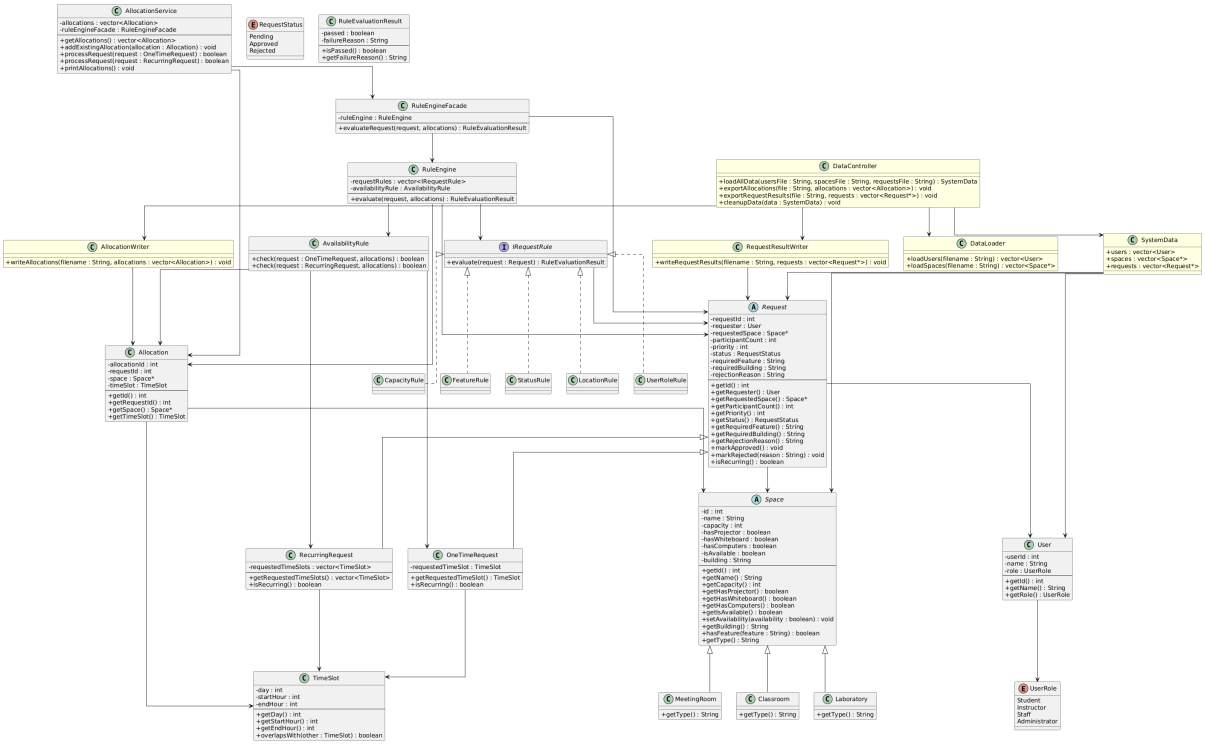

Figure 1: Updated Class Diagram with Data Controller

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. The system now loads all external data through the controller before processing requests, and exports results after processing is completed.

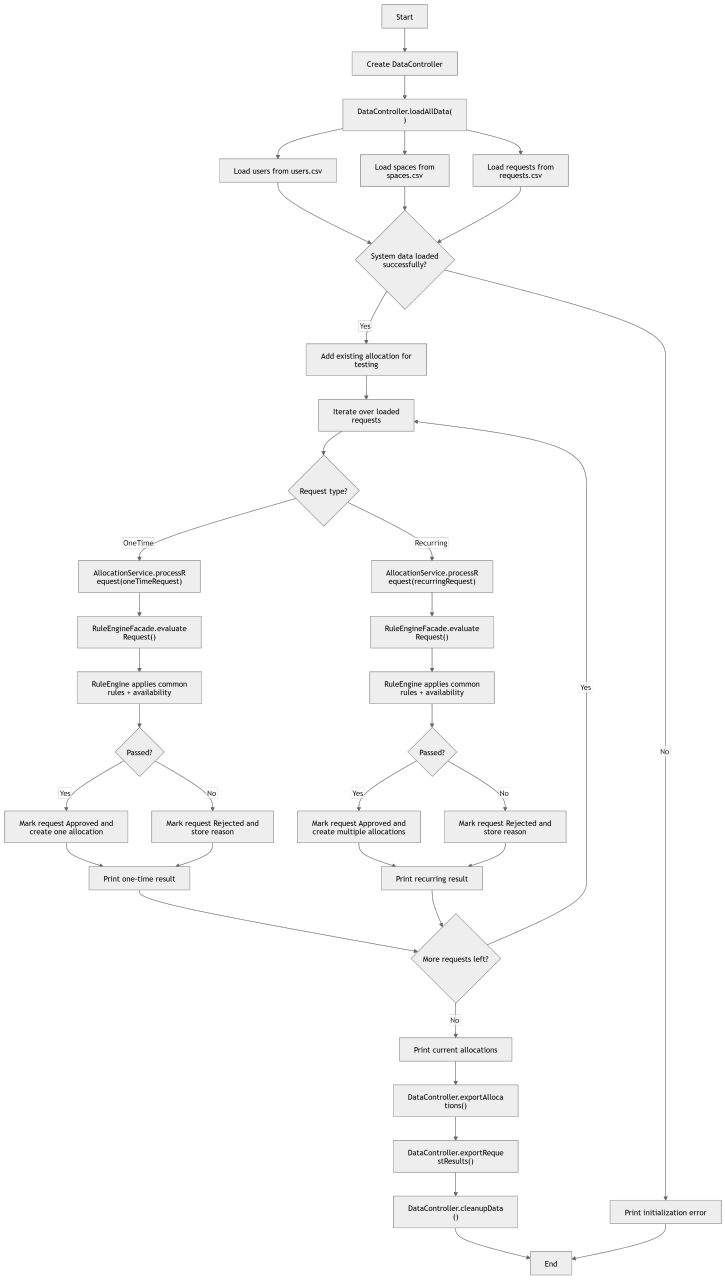

Figure 2: Workflow with External Request Loading and Data Controller

8. Prototype Implementation

The C++ prototype was extended by introducing a `DataController` that coordinates loading users, spaces, and requests from external CSV files. The request model itself was

not changed, but request instances are now created from external input rather than being manually defined in the main program.

The prototype was also extended with a request result export component, allowing the system to save the status and outcome of every processed request, including rejection reasons, to a CSV file.

This iteration improves separation of concerns by reducing the dependency of the main program and internal business logic on file-specific loading details.

9. Summary

This sixteenth implementation iteration improves the architecture of the prototype by externalizing request input and centralizing data access through a dedicated controller. As a result, the internal business logic is less dependent on how data is fetched or exported.

This iteration provides a stronger foundation for future enhancements such as batch request scheduling, alternative data sources, or database-backed persistence.